Experiment 3: Uniform Cost Search (UCS)

Aim

To implement Uniform Cost Search to find the least-cost path between source and destination in a weighted graph.

Algorithm:

1. Use a priority queue ordered by path cost.
2. Insert (0, start, [start]) into the priority queue.
3. While PQ is not empty:
   a. Pop the node with lowest cost.
   b. If it is the goal, return path and cost.
   c. Expand neighbors and push (new_cost, neighbor, new_path).
4. Skip already visited nodes to avoid cycles.

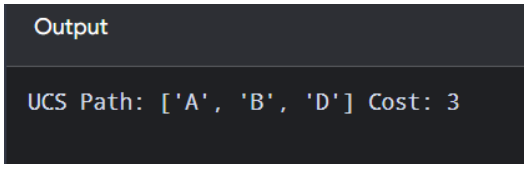
Code:

```
import heapq

def ucs(graph, start, goal):
    pq = [(0, start, [start])]
    visited = set()
    while pq:
        cost, node, path = heapq.heappop(pq)
        if node in visited: continue
        visited.add(node)
        if node == goal:
            return path, cost
        for n, c in graph.get(node, []):
            heapq.heappush(pq, (cost + c, n, path + [n]))
    return None, float('inf')

graph = {'A':[('B',2),('C',5)], 'B':[('D',1)], 'C':[('D',2)], 'D':[]}
path, cost = ucs(graph, 'A', 'D')
print("UCS Path:", path, "Cost:", cost)
```

Sample Output:



Result:

UCS correctly selected the minimum cost path ['A','B','D'] with total cost 3, which is cheaper than the alternative path via C (cost 7).